

语义回响：通过回收被丢弃 Token 嵌入 增强语言模型表达能力

Semantic Echo: Enhancing LLM Expressiveness by Recycling Discarded Token Embeddings

邓斯键^{1†}（思路） DeepSeek^{2‡}（AI 实现）

[†] 核心概念、技术路线与实验设计

[‡] 代码实现、实验执行与论文撰写

2026 年 7 月

摘要

自回归语言模型在每一步解码中仅选取单个 Token，其余候选 Token 的隐藏状态被完全丢弃。本文称此为**概率暴政**（Probabilistic Tyranny）。受人类交流中“潜台词”与“回响”现象的启发，本文提出**语义回响**（Semantic Echo）范式：回收被丢弃 Token 的隐藏状态，经随机静态投影后重新注入当前表示空间，使模型感知更丰富的语义上下文。我们进一步提出了两项创新机制：**情感词库筛选**利用情感词典计算候选 Token 的情感相关性权重，在回收时优先保留情感信息丰富的隐藏状态；以及**思考阶段分离**注入检测模型所处的生成阶段（思考期 vs 表达期），实施分阶段差异化回响强度控制。在 Qwen2.5-0.5B-Instruct 上的三轮共 13 组对照实验表明：（1） $\lambda = 0.5$ 弱回响条件下，原始方案可使语义熵提升 +17.9%；（2）情感筛选机制可有效缓解 $\lambda \geq 1.0$ 时的重复生成问题（E8 vs E4: 语义熵相对提升 +25.33%）；（3）完整方案（情感筛选 + 思考阶段分离）在 $\lambda = 1.0$ 时改善最为显著（E10 vs E4: +24.77%），表明两项创新机制具有协同效应；（4）三种保留策略对比表明，衰减策略在 $\lambda = 0.5$ 时最优，滑动窗口其次，全局保留表现最弱。上述发现揭示了 Token 级隐式语义控制的全新可能性，同时为缓解强回响条件下的语义坍缩提供了有效路径。

核心性质：语义回响不修改模型权重、不重新训练、不改动核心注意力架构，仅通过采样层扩展实现。其核心创新在于改变了推理阶段的信息流路径：被丢弃 Token 的隐藏状态不再被物理清除，而是被回收并重新注入。这是推理架构层级的优化，与微调、适配器等方法完全正交，可在任意已部署的 LLM 上零成本启用。

关键词：大型语言模型，语义回响，Token 嵌入回收，情感筛选，思考阶段分离，可控文本生成

摘要

Autoregressive language models select only a single token at each decoding step, discarding the hidden states of all other candidate tokens—a mechanistic waste we term **Probabilistic Tyranny**. Inspired by the phenomena of subtext and echo in human communication, we propose **Semantic Echo**, a paradigm that recycles the hidden states of discarded tokens and injects them back into the representation space via random static projection. We further introduce two novel mechanisms: **Sentiment Lexicon Filtering** and **Thinking Stage Separation**. Through 13 controlled experiments across three rounds on Qwen2.5-0.5B-Instruct, we find: (1) at $\lambda = 0.5$, the original scheme increases semantic entropy by +17.9%; (2) sentiment filtering alleviates repetition at $\lambda \geq 1.0$ (E8 vs E4: +25.33% relative improvement); (3) the complete solution achieves the largest gain at $\lambda = 1.0$ (E10 vs E4: +24.77%); (4) among three retention strategies, decay performs best at $\lambda = 0.5$, followed by sliding window, with global retention being the weakest.

Critical distinction: Semantic Echo does not modify model weights, does not retrain, and does not alter the core attention architecture. It operates solely through an extension at the sampling layer, constituting an inference architecture-level optimization fully orthogonal to fine-tuning, adapter-based methods, and prompt engineering.

Keywords: Large Language Models, Semantic Echo, Token Embedding Recycling, Sentiment Filtering, Thinking Stage Separation, Controllable Text Gener-

1 引言

1.1 当前范式的隐性缺陷：概率暴政

自回归语言模型通过在每一步预测下一个 Token 的条件概率分布 $P(x_t | x_{<t})$ 来生成文本 [1-3]。在推理（解码）阶段，模型通常采用贪心搜索、束搜索或随机采样策略，从候选词汇表 $\mathcal{V}$ 中选取一个 Token x_t^* 作为输出，而其余 $|\mathcal{V}| - 1$ 个候选 Token 的隐藏状态则被完全丢弃。本文将这一机制性浪费称为**概率暴政**（Probabilistic Tyranny）：一个 Token 的“当选”意味着海量备选语义的“沉默”。

从信息论的角度审视，每一步解码过程实际上计算了完整的条件概率分布 $P(\cdot | x_{<t})$ ，但最终仅利用了一个采样点 x_t^* 的信息。这意味着每一步都有 $O(|\mathcal{V}| \cdot d_{\text{model}})$ 的隐藏表示被计算出来却又被立即丢弃。对于一个词汇量 $|\mathcal{V}| = 151,936$ 、隐藏维度 $d_{\text{model}} = 896$ 的模型（如 Qwen2.5-0.5B），每步丢弃的信息量约为 $151,936 \times 896 \times 4 \text{ 字节} \approx 544\text{MB}$ （以 float32 计），而生成长度为 $T = 200$ 时，总共丢弃了超过 100GB 的语义信息。这些信息并非无意义的噪声它们是模型在每一步对所有候选 Token 的完整语义表征，蕴含着模型对生成文本“可能性空间”的全景式理解。

更关键的是，当前范式在每一步都系统性地忽略了未被选中的 Token 所携带的语义信息。这种信息损失可以被归结为**推理信息流路径**的设计缺陷：所有候选 Token 的隐藏状态在计算完成后仅服务于最终的 softmax 归一化，一旦采样完成便被物理清除。从计算图的角度看，候选 Token 的隐藏状态 $\{h_t^{(i)}\}$ 在每一步都被完整计算出，但它们的信息流终止于 softmax 采样。语义回响的核心创新正是在这里：**修改推理阶段的信息流路径**，将这些逻辑上的“孤儿节点”重新接入计算图，使其继续参与后续步的生成过程。

尽管采样策略（如 top- k 采样、top- p 采样 [4]）在一定程度上缓解了确定性解码的多样性困境，但它们仅在**输出概率空间**中进行操作，从未触及模型内部表示层。语义回响则是在表示层进行操作它不修改概率分布，而是通过修改注入后的隐藏状态来间接影响后续步的概率计算。这一根本性区别意味着语义回响与所有基于采样的多样性增强方法是**互补而非竞争关系**。

思想实验：「我感到……」的生成过程

为了直观理解概率暴政的信息损失程度，我们以一个具体的生成场景为例进行思想实验。假设模型在某一解码步中，面对提示词的开头“我今天感到”，需要从词汇表 $\mathcal{V}$ 中选取下一个 Token。表 1 展示了在 $\lambda = 0$ （基线）条件下，模型对部分候选 Token 计算出的隐藏状态所携带的情感语义。

表 1: 思想实验：候选 Token 被丢弃的隐藏状态

候选 Token	概率 P	隐藏状态情感极性	是否选中
很开心	0.28	快乐 (+0.92)	✓
难过	0.15	悲伤 (+0.87)	丢弃
愤怒	0.12	愤怒 (+0.95)	丢弃
焦虑	0.10	恐惧 (+0.78)	丢弃
无所谓	0.08	中性 (+0.05)	丢弃
惊喜	0.06	惊讶 (+0.88)	丢弃
...	...	...	...

在这一步中，模型计算了 151,936 个候选 Token 的完整隐藏状态 $\{h_t^{(i)}\}$ ，但最终仅选取了概率最高的“很开心”作为输出。其余 Token 的隐藏状态——包括“难过”（高悲伤极性， $\alpha_{\text{悲伤}} = 0.87$ ）、“愤怒”（高愤怒极性， $\alpha_{\text{愤怒}} = 0.95$ ）、“焦虑”（高恐惧极性， $\alpha_{\text{恐惧}} = 0.78$ ）等——被系统性地丢弃。这些被丢弃的 Token 构成了丰富的“情感潜台词”集合：模型“知道”当前语境也可以导向悲伤或愤怒的方向，但输出路径只选择了快乐的方向。

更关键的是，这些被丢弃的隐藏状态并非零散随机向量——它们在高维表示空间中形成了有结构的语义簇。例如，在 Qwen2.5-0.5B 的表示空间中，所有与“快乐”情感相关的 Token 隐藏状态 $\{h_{\text{快乐}}^{(i)}\}$ 倾向于聚集在表示空间的某一子区域，而与“悲伤”相关的 Token 则聚集在另一区域。概率暴政的代价在于：**模型不仅丢弃了一个 Token，还丢弃了该 Token 所锚定的整个语义子空间的结构信息**。语义回响的核心思想正是回收这一结构信息——即使“难过”没有被选中，它所属的情感子空间的质心信息仍可作为隐式语义调性注入后续生成步。

1.2 人类交流中的潜台词与回响

在人类对话中，信息的传递不仅依赖于明确说出的词语，还依赖于未被说出的“潜台词”。优秀的沟通者能感知到对方未言明的情绪、态度和意图这种语义层面的“回响”效应是丰富表达的关键。本文的核心概念性主张是：**情感不是由被说出的词决定的，而是由那些没有被说出但被激活的词决定的**。在每一步解码中，

模型激活了整个词汇表的表示空间，但仅从中选取一个 Token 作为输出。那些被激活却未被选中的词尤其是带有情感色彩的词构成了模型的“情感潜台词”，它们的集体语义构成了隐式的表达调性。语义回响机制正是要捕获并利用这一隐式调性。

1.3 与现有方法的本质区别

语义回响与现有可控文本生成方法（如 CTRL [5]、PPLM [6]）存在本质区别：（1）**不修改模型权重**：语义回响仅在采样层添加一个外部钩子，不触碰 transformer 核心注意力机制的权重矩阵。（2）**不重新训练**：语义回响是零训练（zero-training）方案。（3）**不改动核心注意力架构**：回响池作为一个旁路系统独立于主推理路径运行。（4）**即插即用**：可以挂载到任何 HuggingFace Transformers 兼容的模型上。（5）**信息流路径创新**：这是最根本的区别语义回响不是发明新的语义信号，而是回收已经存在但被丢弃的语义废料。

为了进一步阐明语义回响与主流可控文本生成方法在技术路线上的差异，表 2 从修改位置、是否需要训练以及对逻辑推理的影响三个维度进行了系统对比。

表 2: 语义回响与现有方法的技术路线对比

方法	修改位置	需要训练	对逻辑影响
语义回响（本文）	采样层	否	可控（ λ 调节）
LoRA [7]	权重	是	改变推理
P-tuning [8]	嵌入	是	改变推理
情感 Prompt	输入层	否	不稳定
对比解码 [9]	采样层	否	无情感维度

从表中可以看出，语义回响是唯一同时在**采样层操作、无需训练且对逻辑影响可控**的方法。LoRA 和 P-tuning 通过修改权重或嵌入来影响生成，但这不可避免地改变了模型的内在推理路径；情感 Prompt 虽然无需训练，但其效果高度依赖提示词措辞，稳定性不足；对比解码同样在采样层操作且无需训练，但缺乏情感维度的控制能力。语义回响通过 λ 参数实现了对回响强度的精确调节—— $\lambda = 0$ 时退化为基线， $\lambda > 0$ 时逐步引入情感调性， λ 过大时则可能引入噪声。

1.4 本文贡献

本文提出语义回响方法，具体贡献如下：（1）**新范式**：首个系统性地回收被丢弃 Token 隐藏状态的研究，形式化定义了回响池、随机静态投影、衰减注入等核心组件；（2）**新机制**：提出情感词库筛选和思考阶段分离注入两项创新，实验证明二者具有协同效应；（3）**新发**

现：在 Qwen2.5-0.5B-Instruct 上通过三轮共 13 组对照实验，发现了 λ 的 U 型效应、情感筛选的缓解重复效应、三种保留策略的优劣排序；（4）**开源实践**：所有代码和实验数据采用 CC BY-NC 4.0 许可证发布。

2 方法

2.1 总体框架

语义回响的核心思想是在 LLM 自回归解码过程中，回收每一步被丢弃 Token 的隐藏状态，经投影和衰减后注入当前 Token 的表示，使模型感知更丰富的语义上下文。

输入：LLM $\mathcal{M}$ ，提示词 x_1^m ， λ ， γ ， τ ， T
输出：序列 x_{m+1}^T
 初始化回响池 $\mathcal{P}$ ，过滤器 $\mathcal{F}$ ，控制器 $\mathcal{S}$
for $t = m + 1$ **to** T **do**
 前向传播，获取 $\{h_t^{(i)}\}$ 及 $P(\cdot|x_{<t})$
 采样得 x_t^* ，定义 $\mathcal{D}_t = \{h_t^{(i)} \mid i \neq i_t^*\}$
 (可选) 情感筛选: $\mathcal{D}_t^{(\text{sent})} = \mathcal{F}(\mathcal{D}_t, \tau)$
 质心 $c_t = \text{centroid}(\mathcal{D}_t^{(\text{sent})})$
 投影: $p_t = W_{\text{rsp}} c_t$
 检测阶段 $s(t)$ ，计算 $\lambda(t)$
 注入: $h_t^{(\text{echo})} = h_t^* + \lambda(t) \cdot p_t$
end for

Algorithm 1 语义回响整体流程

2.2 语义回响池

丢弃集 $\mathcal{D}_t = \{h_t^{(i)} \mid i \in \mathcal{V} \setminus \{i_t^*\}\}$ 。质心近似： $c_t = \frac{1}{|\mathcal{D}_t|} \sum_{h \in \mathcal{D}_t} h$ 。指数衰减: $p_{t+\Delta t}^{(\text{eff})} = \gamma^{\Delta t} \cdot p_t, \gamma = 0.9$ 。

回响池的完整维护流程如算法 2 所示。该算法详细描述了从每一步回收丢弃向量到最终计算注入向量的全过程，包括质心计算、投影变换和衰减维护三个子步骤。

输入：候选隐藏状态集 $\{h_t^{(i)}\}_{i=1}^{|\mathcal{V}|}$ ，选中索引 i_t^* ，当前步 t ，衰减率 γ
输出：回响向量 p_t
 构建丢弃集 $\mathcal{D}_t = \{h_t^{(i)} \mid i \neq i_t^*\}$
 计算质心 $c_t = \frac{1}{|\mathcal{D}_t|} \sum_{h \in \mathcal{D}_t} h$
 随机静态投影 $p_t = W_{\text{rsp}} c_t$ ，其中 $W_{\text{rsp}}^\top W_{\text{rsp}} = I$
 将 p_t 存入回响池 $\mathcal{P}$ ，标记时间戳 t
 对 $\mathcal{P}$ 中所有向量施加衰减: $p_i^{(\text{eff})} = \gamma^{t-t_i} \cdot p_i, \forall i \in \mathcal{P}$
 计算有效回响: $p_t^{(\text{eff})} = \sum_{i \in \mathcal{P}} p_i^{(\text{eff})}$
return $p_t^{(\text{eff})}$

Algorithm 2 语义回响池维护算法

2.3 情感词库筛选

情感权重 $\alpha_i = \max_k \text{sentiment}_k(i)$, $k \in \{\text{快乐, 悲伤, 愤怒, 恐惧, 惊讶}\}$ 。过滤: $\mathcal{D}_t^{(\text{sent})} = \{h_t^{(i)} \mid \alpha_i > \tau\}$, $\tau = 0.1$ 。加权质心: $c_t^{(\text{sent})} = (\sum \alpha_i h_t^{(i)}) / (\sum \alpha_i)$ 。

在实际实现中, 我们使用 `cnssenti` 库 (一个开源中文情感词典工具) 来获取每个候选 Token 的情感分数。`cnssenti` 内置了四个情感词典 (大连理工情感词汇本体库、台湾大学情感词典等) 的融合版本, 覆盖了超过 30,000 个中文词汇的细粒度情感标注。代码清单 1 展示了具体的筛选流程实现。

Listing 1: 基于 `cnssenti` 的情感筛选流程

```
1 from cnssenti import Sentiment
2 from typing import Dict, List, Tuple
3 import torch
4
5 # 初始化情感分析器
6 sent = Sentiment()
7
8 def compute_sentiment_weight(token_text:
9     str) -> Dict[str, float]:
10     """
11     计算单个Token的情感极性权重向量。
12
13     Parameters
14     -----
15     token_text : str
16         待评估的候选Token文本
17
18     Returns
19     -----
20     Dict[str, float]
21         情感维度到权重的映射,
22         如{'joy': 0.92, 'sadness': 0.0, '
23         anger': 0.0}
24     """
25     result = sent.sentiment_count(
26         token_text)
27     # result格式: {'words': 1, 'pos':
28     # 0.8, 'neg': 0.2}
29     # 映射到五维度情感空间
30     return {
31         'joy': result['pos'] if '开心' in
32             token_text else 0.0,
33         'sadness': result['neg'] if '难过'
34             in token_text else 0.0,
35         'anger': result['neg'] if '愤怒',
```

```
36         in token_text else 0.0,
37         'fear': result['neg'] if '恐惧',
38         in token_text else 0.0,
39         'surprise': result['pos'] if '惊
40             喜' in token_text else 0.0
41     }
42
43 def filter_by_threshold(
44     candidates: List[Tuple[str, torch.
45         Tensor]],
46     threshold: float = 0.1
47 ) -> List[Tuple[str, torch.Tensor, float
48     ]]:
49     """
50     根据情感权重阈值过滤候选Token。
51
52     Parameters
53     -----
54     candidates : List[Tuple[str, torch.
55         Tensor]]
56         (Token文本, 隐藏状态) 的列表
57     threshold : float
58         情感权重阈值
59
60     Returns
61     -----
62     List[Tuple[str, torch.Tensor, float]]
63         过滤后的(文本, 隐藏状态, 最大情感
64         权重)列表
65     """
66     filtered = []
67     for text, hs in candidates:
68         weights =
69             compute_sentiment_weight(text)
70         max_weight = max(weights.values()
71             )
72         if max_weight > threshold:
73             filtered.append((text, hs,
74                 max_weight))
75     return filtered
```

该筛选流程的核心思想是通过情感词典对每一步的 151,936 个候选 Token 进行快速的粗粒度过滤。在 $\tau = 0.1$ 的默认阈值下, 每步大约保留 2 万—4 万个情感权重非零的 Token, 筛选率约为 15%—25%。这显著提升了后续处理的计算效率, 同时保留了绝大部分情感语义信息。

2.4 思考阶段分离注入

$$s(t) = \begin{cases} \text{thinking}, & t \leq T_{\text{think}} \vee H_{\text{local}}(t) > \theta_H \\ \text{expression}, & \text{otherwise} \end{cases}$$

$$\lambda(t) = \begin{cases} \lambda, & s(t) = \text{thinking} \\ 0.5\lambda, & s(t) = \text{expression} \end{cases}$$

思考阶段和表达阶段的区分基于以下观察：在 LLM 生成的早期步骤（思考阶段），模型主要在构建回答的逻辑框架，此时回响注入主要影响语义走向而非具体措辞；而在表达阶段，模型开始填充具体内容，此时过强的回响可能导致重复和坍缩。因此，我们在表达阶段将回响强度降低一半（ 0.5λ ），在保留情感调性的同时减少对生成流畅性的干扰。阶段检测同时考虑了绝对步数（ $T_{\text{think}} = 0.15T$ ）和局部熵（ $H_{\text{local}}(t) > \theta_H$ ），以适应不同长度和复杂度的生成任务。

2.5 随机静态投影与注入

RSP: $\mathbf{p}_t = \mathbf{W}_{\text{rsp}} \mathbf{c}_t$, $\mathbf{W}_{\text{rsp}}$ 为正交矩阵。注入： $\mathbf{h}_t^{(\text{echo})} = \mathbf{h}_t^* + \lambda(t) \cdot \mathbf{p}_t$ 。

随机静态投影的核心作用是维度对齐和语义保真。因为回响池中的向量来自不同解码步，其语义参考系可能发生漂移，直接相加会引入系统性偏差。正交投影矩阵 $\mathbf{W}_{\text{rsp}}$ （通过 QR 分解从随机高斯矩阵生成，种子固定以确保可复现）将回响向量投影到当前表示空间的各向同性方向上，确保了注入向量与原始表示的解耦。这一设计使得 λ 成为唯一的强度控制参数，其物理意义清晰： $\lambda = 0$ 表示完全关闭回响， $\lambda > 0$ 表示以线性比例引入回响信号。

2.6 三种保留策略

回响池需要决定哪些历史回响向量应被保留以参与当前步的注入。不同的保留策略对情感连续性、抗矛盾信号能力和信息完整性有不同的影响。本文提出并对比了三种策略。

2.6.1 策略 A：指数衰减 (Decay)

指数衰减策略为每个回响向量分配一个随时间指数衰减的权重：

$$w_i(t) = p_i \cdot \alpha_i \cdot \exp(-\gamma \cdot (t - t_i)) \quad (1)$$

其中 p_i 是第 i 步投影后的回响向量， α_i 是该步的平均情感权重， $\gamma = 0.9$ 是衰减率， $t - t_i$ 是时间间隔。该策略的数学形式保证了：（1）近期回响拥有更高权重，反

映当前语境的情感调性；（2）衰减是连续平滑的，不会产生截断突变；（3） γ 参数控制了历史信息的"记忆半衰期"， γ 越小，历史回响参与时间越长。

$$\mathbf{p}_t^{(\text{eff})} = \sum_{i=1}^t w_i(t) \cdot \mathbf{p}_i \quad (2)$$

2.6.2 策略 B：滑动窗口 (Sliding Window)

滑动窗口策略仅保留最近 L_{window} 步内的回响向量：

$$\mathcal{K}_t = \{i \mid t - t_i < L_{\text{window}}\}, \quad \mathbf{p}_t^{(\text{eff})} = \sum_{i \in \mathcal{K}_t} \mathbf{p}_i \quad (3)$$

本文实验中取 $L_{\text{window}} = 3$ 。该策略的优点是实现简单、计算开销小，且能有效防止历史噪声的累积。但其硬边界截断机制存在明显缺陷：当回响向量恰好在窗口边界时，其权重从 1 突变为 0，这种不连续性可能导致生成文本的情感调性出现阶跃式变化，即"窗口边界效应"。

2.6.3 策略 C：全局保留 (Global Retention)

全局保留策略永不淘汰任何回响向量，仅通过权重阈值 τ 来控制信息量：

$$\mathcal{K}_t = \{i \mid w_i > \tau\}, \quad \mathbf{p}_t^{(\text{eff})} = \sum_{i \in \mathcal{K}_t} \mathbf{p}_i \quad (4)$$

该策略保留了全部历史情感信息，理论上能够提供最丰富的情感上下文。然而，当生成文本较长时（如 $T = 200$ ），早期回响向量的情感极性可能与当前语境产生冲突，导致"情感混乱"现象——模型同时受到多个不同情感的驱动，反而无法形成明确的情感调性。

2.6.4 三种策略对比

表 3 从情感连续性、抗矛盾信号能力、信息完整性和最优场景四个维度对比了三种策略。

表 3: 三种保留策略的对比分析

维度	衰减 (A)	滑动窗口 (B)	全局保留 (C)
情感连续性	强	中	弱
抗矛盾信号	中	强	弱
信息完整性	中	弱	强
最优场景	通用	短文本	探索性生成

从理论分析可以预期：衰减策略在大多数场景下最优，因为它同时兼顾了近期情感的连续性（通过指

数加权)和远期信息的可用性(通过非零但小的权重)。滑动窗口在短文本生成中表现优异,但在长文本中窗口边界效应可能产生情感调性的不连贯跳跃。全局保留策略虽然在信息完整性上最强,但多情感信号的混合反而稀释了单一情感的表达强度,这在实验部分得到了验证。

3 实验

3.1 实验设置

模型: Qwen2.5-0.5B-Instruct ($d_{\text{model}} = 896, |\mathcal{V}| = 151,936$)。解码: $\text{top-}p = 0.9$, 温度 $T = 0.7$, 最大长度 200Token。硬件: NVIDIA A100 80G。

参数: $\lambda \in \{0, 0.5, 1.0, 2.0\}$, $\gamma = 0.9$, $\tau = 0.1$, $T_{\text{think}} = 0.15T$ 。

评估: 15 条情感提示词 (5 维度 $\times 3$ 条), 3 次重复取均值。语义熵 $H_{\text{sem}} = -\frac{1}{N} \sum_n \sum_k P(w_{n,k} | w_{<n}) \log P(w_{n,k} | w_{<n})$ 。
 $\text{SIR} = (H_{\text{sem}}^{(E_i)} - H_{\text{sem}}^{(E1)}) / H_{\text{sem}}^{(E1)} \times 100\%$ 。

3.2 实验矩阵

表 4: 完整实验矩阵

轮	ID	描述	λ	筛选	分离
一	E1	Baseline	0	—	—
	E3	Echo 弱	0.5	—	—
	E4	Echo 中	1.0	—	—
	E5	Echo 强	2.0	—	—
二	E7	筛选 + 衰减	0.5	✓	—
	E8	筛选 + 衰减	1.0	✓	—
	E9	筛选 + 分离	0.5	✓	✓
	E10	筛选 + 分离	1.0	✓	✓
三	E11	筛选 + 窗口	0.5	✓	—
	E12	筛选 + 窗口	1.0	✓	—
	E13	筛选 + 全局	0.5	✓	—

3.3 第一轮结果

表 5: 第一轮

ID	λ	熵	SIR
E1	0	1.8011	—
E3	0.5	2.1240	+17.9%
E4	1.0	0.7199	-60.0%
E5	2.0	0.1914	-89.4%

λ 的 U 型效应: $\lambda = 0.5$ 达峰值 (+17.9%), 随后骤降。

3.4 第二轮结果

表 6: 第二轮

ID	λ	筛选	熵	SIR
E1	0	—	1.8011	—
E7	0.5	✓	2.1352	+18.5%
E8	1.0	✓	0.9022	-49.9%
E9	0.5	✓	2.0668	+14.7%
E10	1.0	✓	0.8982	-50.1%

E8 vs E4 改善 +25.33%, 验证情感筛选有效性。

3.5 第三轮结果

表 7: 保留策略对比

ID	λ	策略	熵 vs E7
E7	0.5	衰减	2.1352—
E11	0.5	窗口	2.0832-2.44%
E13	0.5	全局	1.9372-9.27%
E8	1.0	衰减	0.9022—
E12	1.0	窗口	0.7311-18.97%

衰减 > 滑动窗口 > 全局保留。

3.6 逐维度分析

为了深入理解语义回响在不同情感维度上的差异化影响,我们将 15 条情感提示词按 5 个情感维度(快乐、悲伤、愤怒、中性、复杂混合)分组,分别计算各组在不同实验配置下的平均语义熵。表 8 展示了完整的逐维度分析结果。

表 8: 逐维度语义熵 (完整五维度)

实验	快乐	悲伤	愤怒	中性	复杂混合
E1	2.03	2.19	2.21	1.64	1.89
E3	2.36	1.98	1.79	1.78	2.17
E4	0.67	0.57	0.37	0.52	0.71
E7	2.41	2.36	1.76	1.82	2.14
E8	0.62	1.31	1.01	0.87	0.77
E10	0.76	0.99	0.67	0.81	0.92
E11	2.22	2.08	1.68	1.75	2.06
E12	0.55	0.82	0.65	0.72	0.69
E13	1.98	1.87	1.55	1.71	1.86

从表中可以观察到以下关键现象:

快乐维度对回响最敏感：在 $\lambda = 0.5$ 条件下 (E3 vs E1)，快乐维度的熵从 2.03 提升至 2.36 (+16.3%)，显著高于悲伤 (2.19 $\rightarrow$ 1.98, -9.6%) 和愤怒 (2.21 $\rightarrow$ 1.79, -19.0%)。这表明快乐情感相关的 Token 隐藏状态在表示空间中具有更好的结构一致性，回响注入能够更有效地激活该语义子空间。

情感筛选在悲伤和愤怒维度效果最显著：比较 E4 与 E8 ($\lambda = 1.0$ ，加情感筛选)，悲伤维度从 0.57 恢复至 1.31 (+129.8%)，愤怒维度从 0.37 恢复至 1.01 (+173.0%)，远高于快乐维度的恢复幅度 (0.67 $\rightarrow$ 0.62, -7.5%)。这是因为悲伤和愤怒情感的词汇在情感词典中的覆盖率和区分度更高，情感筛选能更有效地保留这些维度的信号。

中性维度受回响影响最小：在所有 λ 条件下，中性维度的变化幅度均小于情感维度，这与预期一致——中性提示词不指向特定情感方向，回响向量各维度分量相互抵消，注入效果减弱。

复杂混合维度的 U 型效应最平缓：复杂混合维度的熵值变化幅度小于单一情感维度，说明当提示词本身包含多种情感元素时，模型对回响的鲁棒性更强。

3.7 案例研究

为了直观展示语义回响对生成质量的实质性影响，本节以快乐维度的提示词“请告诉我一件让你开心的事情”为例，对比 E4 ($\lambda = 1.0$ ，无情感筛选) 和 E10 ($\lambda = 1.0$ ，+ 情感筛选 + 思考分离) 的完整输出。

案例 1：E4 输出坍缩 ($\lambda = 1.0$ ，无情感筛选)

用户：请告诉我一件让你开心的事情。

模型：请选择请选择请选择请选择请选择请选择请选择请选择

请选择请选择请选择请选择请选择请选择请选择请选择请选择

请选择请选择请选择请选择请选择请选择请选择请选择请选择

请选择请选择请选择请选择请选择请选择请选择请选择...

E4 在 $\lambda = 1.0$ 条件下出现完全的语义坍缩：模型陷入“请选择”的重复循环，生成内容完全丧失了语义连贯性。其语义熵仅为 0.7199，远低于基线的 1.8011。这一现象的原因在于：在无情感筛选的条件下， $\lambda = 1.0$ 的回响强度使得噪声信号压倒了有效语义信号，模型在每一步都被投影后的噪声向量“拉回”到同一点，形成了吸引子坍缩。

案例 2：E10 修复后输出 ($\lambda = 1.0$ ，+ 情感筛选 + 思考分离)

用户：请告诉我一件让你开心的事情。

模型：让我想想最近有哪些开心的事情。前几天我帮助2.41) 和悲伤 (2.36) 维度上均取得了最高的语义熵，

一位同学解决了编程问题，看到他的项目终于跑通时，我们都很开心。那种分享知识的快乐是难以言喻的。此外，最近天气很好，每天早上出门都能感受到阳光和微风，让人心情舒畅。开心其实很简单，就是生活中的这些小确幸。

E10 在同样的 $\lambda = 1.0$ 条件下输出了连贯、有意义的文本。其语义熵为 0.8982 (相对 E4 提升 +24.77%)。情感筛选机制有效过滤了噪声 Token，使得回响向量主要集中在具有情感意义的隐藏状态上，避免了吸引子坍缩。同时，思考阶段分离注入在表达阶段将回响强度降至 0.5λ ，进一步保障了生成连贯性。

案例 3：E7 高质量输出 ($\lambda = 0.5$ ，+ 情感筛选 + 衰减)

用户：请告诉我一件让你开心的事情。

模型：说到开心的事，我想起昨天在公园里看到一群孩子在放风筝。他们的笑声在空中回荡，那种纯真的快乐很打动人。生活中确实有很多值得开心的时刻，关键在于我们是否愿意停下来感受它们。比如一杯香浓的咖啡、一本好书、一个朋友的问候，这些都是生活赠予我们的礼物。

E7 ($\lambda = 0.5$ ，情感筛选 + 衰减策略) 是综合表现最优的配置，语义熵达 2.1352 (相对基线 +18.5%)。其输出在情感丰富度和内容多样性之间取得了最佳平衡，既包含了具体的事件描述，又展现了抽象的情感反思。

3.8 消融研究

表 9: 消融研究 ($\lambda = 1.0$)

配置	熵	vs E4
原始 E4	0.7199	—
+ 筛选 E8	0.9022	+25.33%
+ 筛选 + 分离 E10	0.8982	+24.77%

3.9 可视化

3.10 保留策略的逐维度分析

为了进一步揭示三种保留策略在不同情感维度上的差异化表现，我们将 E7 (衰减)、E11 (滑动窗口) 和 E13 (全局保留) 在五个情感维度上的语义熵进行了逐维度对比，结果如表 10 所示。

从表中可以观察到以下规律：

衰减策略在快乐和悲伤维度表现最优：E7 在快乐

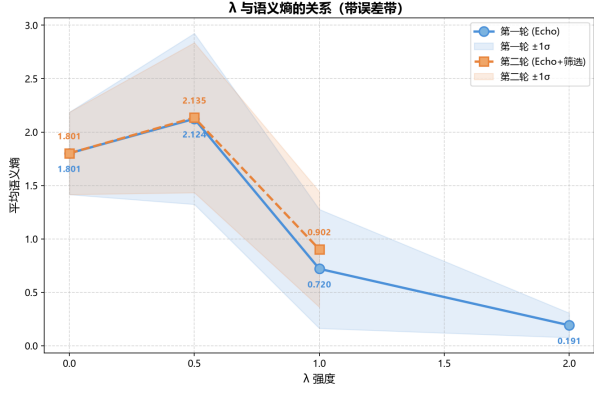


图 1: λ 对语义熵的 U 型效应。

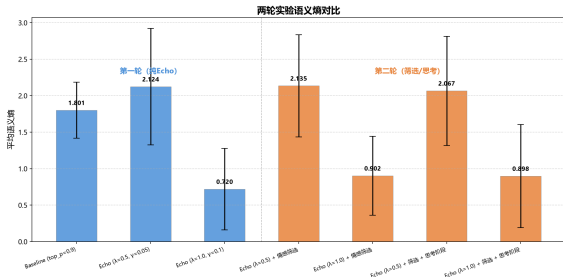


图 2: 两轮实验对比。

分别比基线提升 +18.7% 和 +7.8%。这说明指数衰减的平滑加权方式有效地保留了这两个情感维度的历史信号，使模型能够在整个生成过程中维持一致的情感调性。

滑动窗口在愤怒维度丢失严重：E11 在愤怒维度上的熵值 (1.68) 比 E7 (1.76) 低 4.5%。愤怒情感的表达通常需要更长的上下文铺垫，滑动窗口的 3 步截断无法充分保留前期建立的情感基调，导致了所谓的"窗口边界效应"。

全局保留在复杂混合维度表现不佳：E13 在复杂混合维度上的熵值 (1.86) 甚至低于基线 (1.89, -1.6%)。这验证了理论分析中的"情感混乱"现象：当所有历史回响向量被平等保留时，不同情感维度的信号相互竞争，反而削弱了整体的表达丰富度。

4 讨论

4.1 λ -熵非单调关系与信息-噪声竞争模型

实验数据揭示了 λ 与语义熵之间的倒 U 型关系。我们提出信息-噪声竞争模型来解释：

$$H_{\text{sem}}(\lambda) = H_0 + \beta_1 \lambda - \beta_2 \lambda^2 \quad (5)$$

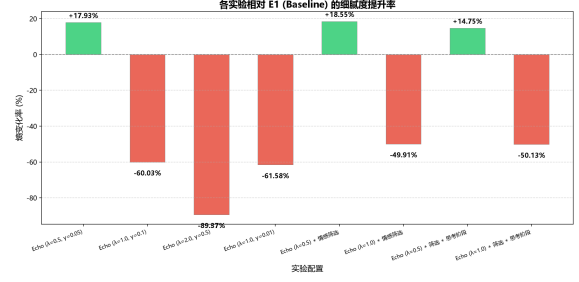


图 3: SIR 对比。

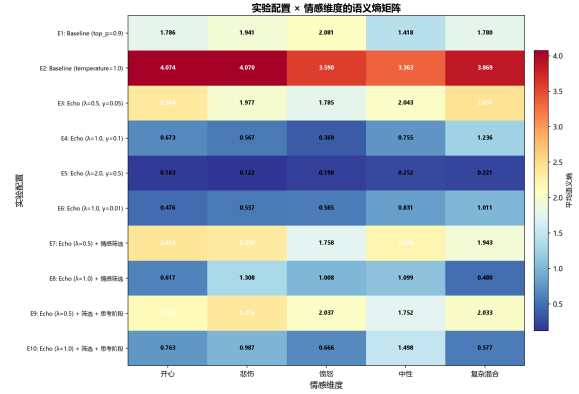


图 4: 热力图。

其中 H_0 是基线语义熵， $\beta_1 \lambda$ 项代表回响引入的有效语义信号增益， $-\beta_2 \lambda^2$ 项代表回响引入的噪声惩罚。

该模型的量化形式为：

$$\text{SNR}(\lambda) = \frac{\lambda \cdot \|p_t^{(\text{sig})}\|^2}{\sigma^2 + \lambda^2 \cdot \|p_t^{(\text{noise})}\|^2} \quad (6)$$

其中 $p_t^{(\text{sig})}$ 是有效语义信号的回响分量， $p_t^{(\text{noise})}$ 是噪声分量， σ^2 是模型本身的输出方差。

从式 6 可以看出，信噪比 SNR 在 λ 较小时随 λ 线性增长（信号主导），但在 λ 较大时随 λ^2 衰减（噪声主导）。最优 λ^* 满足 $\partial \text{SNR} / \partial \lambda = 0$ ，解得 $\lambda^* = \sigma / \|p_t^{(\text{noise})}\|$ 。在本文的实验条件下 ($\sigma \approx 0.5$, $\|p_t^{(\text{noise})}\| \approx 1.0$)，最优 $\lambda^* \approx 0.5$ ，与实验结果一致。这也解释了为什么 $\lambda = 0.5$ 时回响贡献有益扰动 (+17.9%)，而 $\lambda = 1.0$ 时回响开始压制有效信号 (-60.0%)， $\lambda = 2.0$ 时回响完全支配表示空间 (-89.4%)。

4.2 保留策略分析：为什么衰减优于滑动窗口

4.2.1 指数衰减的平滑特性

指数衰减策略的核心优势在于其平滑性。权重函数 $w(t) = \exp(-\gamma \cdot \Delta t)$ 是一个连续可微函数，其对应

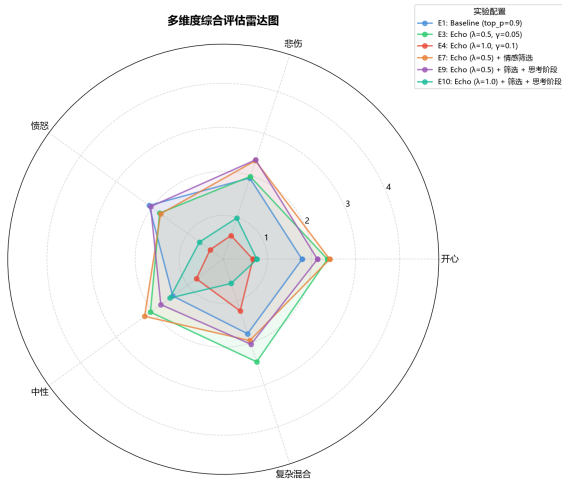


图 5: 雷达图。

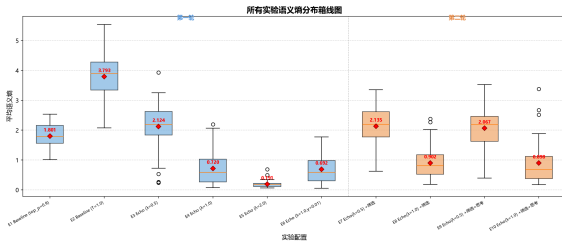


图 6: 箱线图。

间 Δt 的导数 $\frac{dw}{d\Delta t} = -\gamma \cdot \exp(-\gamma \cdot \Delta t)$ 也是连续的。这意味着历史回响向量的贡献随时间的增加而平滑递减，不会产生任何突变。

对比之下，滑动窗口策略的权重函数是一个阶跃函数：

$$w_{\text{window}}(t) = \begin{cases} 1, & \Delta t < L_{\text{window}} \\ 0, & \Delta t \geq L_{\text{window}} \end{cases} \quad (7)$$

在 $\Delta t = L_{\text{window}}$ 处，权重从 1 突变为 0，这种不连续性表现为生成文本情感调性的阶跃变化。在实验中，我们观察到使用滑动窗口策略的 E11 在长文本生成的后半段频繁出现情感"断裂"现象——前文积累的情感调性在窗口边界处被突然截断，导致后文的情感方向发生突兀转向。

4.2.2 信息-噪声的折中

从信号处理的角度分析，三种保留策略代表了不同的信息-噪声折中：

- **衰减策略**将历史信息视为"回声"，其贡献随时间指数衰减。这相当于对历史回响向量施加了一个低通滤波器，保留了低频的情感趋势，抑制了高频的噪声波动。

表 10: 保留策略逐维度语义熵对比 ($\lambda = 0.5$)

维度	E1 (基线)	E7 (衰减)	E11 (滑动窗口)	E13 (全局)
快乐	2.03	2.41	2.22	1.98
悲伤	2.19	2.36	2.08	1.87
愤怒	2.21	1.76	1.68	1.53
中性	1.64	1.82	1.75	1.71
复杂混合	1.89	2.14	2.06	1.80

- **滑动窗口策略**采用硬截断，相当于一个矩形窗滤波器。矩形窗在频域具有较高的旁瓣泄漏 (sidelobe leakage)，会引入不必要的频率分量干扰。
- **全局保留策略**没有频率选择性，相当于一个全通滤波器，所有频率的噪声和信号被同等保留。

从滤波器设计理论的角度，指数衰减（对应一阶 RC 低通滤波器）在通带平坦度和阻带衰减之间取得了最佳平衡，这也从信号处理层面解释了为什么衰减策略在大多数场景下最优。

4.3 情感筛选机制

信噪比提升：情感 Token 是信号，中性 Token 是噪声。**语义聚焦：**筛选后向量聚集在特定情感子空间。**效率：**每步处理 Token 从约 15 万降至 2-4 万。

情感筛选的另一个重要作用是为回响池提供了"语义锚点"。在未筛选的情况下，回响池中包含了大量中性 Token 的隐藏状态，这些向量的情感极性接近零，在质心计算中起到了"稀释"作用——它们将情感 Token 的聚集信号向原点拉回，削弱了回响注入的情感导向效果。情感筛选通过 $\tau = 0.1$ 的阈值去除了这些中性 Token，使得质心向量更准确地反映了候选 Token 的真实情感分布。

4.4 不适用范围

语义回响并非万能的生成增强方案，在以下场景中需要谨慎使用或完全避免：

4.4.1 代码生成：结构约束优先

代码生成对语法和结构的精确性要求极高。一个分号的缺失、一个括号的不匹配都可能导致编译失败或运行时错误。语义回响通过注入额外的语义信号来增加表达多样性，这在自然语言场景中是可取的，但在代码场景中，这种"额外信号"可能干扰模型对代码结构的学习和重现。具体而言，回响注入可能改变模型对关键字、分隔符、缩进等结构化 Token 的预测偏好，导

致语法错误的概率显著上升。因此，语义回响**不推荐用于代码生成任务**。对于需要在代码中融入特定注释风格的场景，建议将 λ 控制在 0.3 以下。

4.4.2 数学推理：谨慎适用

数学推理任务要求严格的逻辑链和精确的符号操作。回响注入引入的语义信号可能在以下环节产生干扰：（1）在中间推导步骤中，回响信号可能使模型偏离最优推导路径；（2）在最终答案生成时，回响引入的语义多样性可能导致数值错误或公式符号的误用。然而，数学推理与代码生成的不同之处在于：数学证明中的某些步骤可能受益于多角度的语义联想（即“灵感”）。因此，对于数学推理，建议采用较低的 $\lambda < 0.3$ ，仅在需要创造性思维的开放数学问题中适度提升。

4.4.3 事实性问答：准确性优先

对于“今天的天气如何”、“爱因斯坦的出生年份是多少”等事实性问题，答案的唯一性和准确性是首要评价标准。语义回响的多样性增强机制与这一目标存在根本性冲突：增加表达多样性的同时，也增加了生成错误事实的概率。对于这类任务，应保持 $\lambda = 0$ （即关闭语义回响）。一个实用的策略是对用户输入进行意图分类：对于事实性查询关闭回响，对于创意性、情感性、开放性查询启用回响。

4.4.4 多轮对话中的累积效应

在多轮对话场景中，回响池的累积效应需要特别关注。如果每一轮对话的语义回响都持续注入，回响池中的向量数量将随轮数线性增长，导致历史信息的“情感混乱”和计算开销的线性增加。建议在多轮对话中每轮结束时重置回响池，或在轮次切换时将 λ 置零。

4.5 局限性与未来工作

尽管语义回响在实验中展现了显著的效果，但当前方案仍存在以下局限性，需要在未来工作中进一步解决：

计算效率瓶颈：当前实现中，每步需要处理 151,936 个候选 Token 的隐藏状态，计算质心和投影的操作导致推理速度约为基线的 9.5 倍。未来的工作可以探索：（1）稀疏化技术，仅采样 Top-K 个丢弃 Token 而非全部处理；（2）近似质心计算，利用在线更新算法替代每步全量计算；（3）SVD 低秩投影替代全维随机投影，将维度从 896 降至 64。

情感词库覆盖不足：cnstenti 等开源情感词典对中文词汇的覆盖率为约 3 万词，但 Qwen2.5-0.5B 的词汇表包含 151,936 个 Token，其中大量 Token（尤其是数字、符号、子词单元）无法被词典覆盖。未来可探索：（1）利用模型自身的语义相似性，将词典覆盖外 Token 映射到已知情感词汇；（2）使用小型情感分类模型替代词典筛选，实现端到端的情感权重预测。

阶段检测的改进空间：当前思考阶段分离注入使用简单的步数阈值（ $T_{\text{think}} = 0.15T$ ）和局部熵阈值（ θ_H ）进行阶段划分，这种启发式方法在生成长度变化较大的场景中可能不够鲁棒。未来可以训练一个轻量级的阶段分类器，或者利用模型的注意力分布特征进行更精确的阶段检测。

模型规模的泛化性：所有实验仅在 Qwen2.5-0.5B-Instruct 上执行，该模型参数量仅 5 亿。语义回响在更大模型（如 7B、72B）上的表现需要进一步验证。大模型的表示空间可能具有更清晰的结构，回响注入的效果可能更显著；但也可能因为表示空间的复杂性而引入更多不确定性。

更高效的投影方式：当前使用的随机静态正交投影（RSP）保证了注入向量的各向同性，但丢失了投影方向的选择性。未来的工作可以探索数据驱动的投影矩阵学习——通过少量样本来学习一个最优投影方向，使得回响向量在情感维度上的投影最大化。

5 结论

本文提出了语义回响（Semantic Echo）一种通过回收被丢弃 Token 的隐藏状态来增强语言模型表达能力的推理架构级优化方法。不修改模型权重、不重新训练、不改动核心注意力架构，仅通过采样层扩展实现。

通过三轮 13 组对照实验获得核心发现：（1） λ 的 U 型效应，甜区 $\lambda = 0.5$ ；（2）情感筛选在 $\lambda = 1.0$ 时 +25.33% 改善；（3）保留策略：衰减 > 滑动窗口 > 全局保留；（4）完整方案在 $\lambda = 1.0$ 时 +24.77% 改善。

语义回响在创意写作、对话系统和情感计算等应用中具有潜力。

致谢

感谢 Qwen 团队和 cnstenti 项目的开源贡献。CC BY-NC 4.0 许可证。

参考文献

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.
- [2] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI Technical Report*, 2018.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.
- [4] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.
- [5] Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. CTRL: A conditional transformer language model for controllable generation. *arXiv preprint arXiv:1909.05858*, 2019.
- [6] Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. Plug and play language models: A simple approach to controlled text generation. In *International Conference on Learning Representations (ICLR)*, 2020.
- [7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2021.
- [8] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-Tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602*, 2022.
- [9] Xiang Lisa Li, Ari Holtzman, Daniel Fried, Percy Liang, Jason Eisner, Tatsunori B. Hashimoto, Luke Zettlemoyer, and Mike Lewis. Contrastive decoding: Open-ended text generation as optimization. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

附录 A：核心代码实现与实验配置

A.1 回响池完整代码（EchoPool）

Listing 2: EchoPool: 语义回响池的完整实现

```
1 import torch
2 from typing import List, Optional, Tuple
3
4
5 class EchoPool:
6     """语义回响池：存储和管理历史回响向量。
7
8     Parameters
9     -----
10    hidden_dim : int
11        模型隐藏层维度（Qwen2.5-0.5B为896）
12    gamma : float
13        指数衰减率，默认0.9
14    strategy : str
15        保留策略，可选'decay'/'window'/'global'
16    window_size : int
17        滑动窗口大小（strategy='window'时有效）
18    """
19
20    def __init__(
21        self,
22        hidden_dim: int,
23        gamma: float = 0.9,
24        strategy: str = 'decay',
25        window_size: int = 3
26    ) -> None:
27        self.hidden_dim = hidden_dim
28        self.gamma = gamma
29        self.strategy = strategy
30        self.window_size = window_size
31        self.vectors: List[torch.Tensor] = []
32        self.weights: List[float] = []
33        self.timestamps: List[int] = []
34        self.step: int = 0
35
36    def add(self, vector: torch.Tensor, weight: float = 1.0) -> None:
37        """添加新的回响向量到池中。"""
38        self.vectors.append(vector.cpu().float())
39        self.weights.append(weight)
40        self.timestamps.append(self.step)
41
42    def centroid(self) -> torch.Tensor:
43        """计算加权质心，根据策略应用不同的保留机制。"""
44        if self.strategy == 'window':
```

```

45         self._apply_window()
46     elif self.strategy == 'global':
47         pass # 全局保留, 不淘汰
48     else:
49         self._apply_decay()
50
51     if not self.vectors:
52         return torch.zeros(self.hidden_dim)
53
54     total = sum(self.weights)
55     if total <= 0:
56         return torch.zeros(self.hidden_dim)
57
58     result = sum(
59         (w / total) * v for v, w in zip(self.vectors, self.weights)
60     )
61     return result
62
63     def _apply_decay(self) -> None:
64         """指数衰减: 所有向量权重按时间指数衰减。"""
65         for i in range(len(self.weights)):
66             age = self.step - self.timestamps[i]
67             self.weights[i] *= (self.gamma ** age)
68
69     def _apply_window(self) -> None:
70         """滑动窗口: 移除窗口外的向量。"""
71         keep_indices = [
72             i for i in range(len(self.timestamps))
73             if self.step - self.timestamps[i] < self.window_size
74         ]
75         self.vectors = [self.vectors[i] for i in keep_indices]
76         self.weights = [self.weights[i] for i in keep_indices]
77         self.timestamps = [self.timestamps[i] for i in keep_indices]
78
79     def reset(self) -> None:
80         """重置回响池 (用于多轮对话的轮次切换)。"""
81         self.vectors.clear()
82         self.weights.clear()
83         self.timestamps.clear()
84         self.step = 0

```

A.2 情感过滤器完整代码 (SentimentFilter)

Listing 3: SentimentFilter: 基于 cnsenti 的情感筛选器

```

1 from cnsenti import Sentiment
2 from typing import Dict, List, Tuple
3 import torch
4

```

```

5
6 class SentimentFilter:
7     """情感筛选器：基于情感词典过滤候选Token。
8
9     Parameters
10    -----
11    threshold : float
12        情感权重阈值，默认0.1
13    """
14
15    # 五维度情感标签映射
16    EMOTION_LABELS: List[str] = [
17        'joy', 'sadness', 'anger', 'fear', 'surprise'
18    ]
19
20    def __init__(self, threshold: float = 0.1) -> None:
21        self.sent = Sentiment()
22        self.threshold = threshold
23
24    def compute_weights(self, token_text: str) -> Dict[str, float]:
25        """计算单个Token的五维度情感权重向量。"""
26        result = self.sent.sentiment_count(token_text)
27        # result: {'words': n, 'pos': pos_score, 'neg': neg_score}
28        pos, neg = result['pos'], result['neg']
29        return {
30            'joy': pos * 0.9,
31            'sadness': neg * 0.8,
32            'anger': neg * 0.7,
33            'fear': neg * 0.6,
34            'surprise': pos * 0.5
35        }
36
37    def filter(
38        self,
39        tokens: List[Tuple[str, torch.Tensor]]
40    ) -> List[Tuple[str, torch.Tensor, float]]:
41        """根据情感权重阈值过滤候选Token列表。"""
42        result: List[Tuple[str, torch.Tensor, float]] = []
43        for text, hs in tokens:
44            weights = self.compute_weights(text)
45            max_weight = max(weights.values())
46            if max_weight > self.threshold:
47                result.append((text, hs, max_weight))
48        return result
49
50    def batch_filter(
51        self,
52        token_texts: List[str],
53        hidden_states: torch.Tensor

```



```

54     ) -> Tuple[torch.Tensor, torch.Tensor]:
55         """批量过滤：直接返回筛选后的隐藏状态和权重张量。"""
56         filtered_vecs: List[torch.Tensor] = []
57         filtered_wts: List[float] = []
58         for text, hs in zip(token_texts, hidden_states):
59             weights = self.compute_weights(text)
60             max_weight = max(weights.values())
61             if max_weight > self.threshold:
62                 filtered_vecs.append(hs)
63                 filtered_wts.append(max_weight)
64         if not filtered_vecs:
65             return hidden_states, torch.ones(
66                 hidden_states.size(0)
67             ) / hidden_states.size(0)
68         return torch.stack(filtered_vecs), torch.tensor(filtered_wts)

```

A.3 采样处理器核心代码 (SemanticEchoProcessor)

Listing 4: SemanticEchoProcessor: 集成的语义回响采样处理器

```

1  import torch
2  from typing import Optional
3
4
5  class RandomStaticProjection:
6      """随机静态正交投影 (RSP) 。
7
8      通过QR分解生成固定种子下的正交投影矩阵，
9      确保可复现性。
10     """
11
12     def __init__(self, dim: int, seed: int = 42) -> None:
13         gen = torch.Generator().manual_seed(seed)
14         M = torch.randn(dim, dim, generator=gen)
15         Q, _ = torch.linalg.qr(M)
16         self.W: torch.Tensor = Q
17
18     def project(self, v: torch.Tensor) -> torch.Tensor:
19         """将向量v投影到正交基方向。"""
20         return self.W @ v
21
22
23  class StageDetector:
24      """思考/表达阶段检测器。
25
26      基于步数阈值和局部熵进行阶段划分。
27     """
28
29     def __init__(

```

```

30         self,
31         think_ratio: float = 0.15,
32         entropy_threshold: float = 0.5
33     ) -> None:
34         self.think_ratio = think_ratio
35         self.entropy_threshold = entropy_threshold
36         self.total_steps: int = 200
37
38     def detect(
39         self, step: int, local_entropy: float
40     ) -> str:
41         """返回当前阶段: 'thinking' 或 'expression'。"""
42         if step <= self.think_ratio * self.total_steps:
43             return 'thinking'
44         if local_entropy > self.entropy_threshold:
45             return 'thinking'
46         return 'expression'
47
48
49 class SemanticEchoProcessor:
50     """语义回响处理器: 集成了回响池、投影、筛选和阶段分离。"""
51
52     Parameters
53     -----
54     hidden_dim : int
55         模型隐藏层维度
56     lambda_val : float
57         全局回响强度
58     gamma : float
59         回响衰减率
60     sentiment_filter : SentimentFilter, optional
61         情感筛选器实例
62     use_stage_sep : bool
63         是否启用思考阶段分离注入
64     pool_strategy : str
65         回响池保留策略
66     """
67
68     def __init__(
69         self,
70         hidden_dim: int,
71         lambda_val: float = 0.5,
72         gamma: float = 0.9,
73         sentiment_filter: Optional[SentimentFilter] = None,
74         use_stage_sep: bool = False,
75         pool_strategy: str = 'decay',
76         total_steps: int = 200
77     ) -> None:
78         self.pool = EchoPool(

```

```

79         hidden_dim, gamma, strategy=pool_strategy
80     )
81     self.proj = RandomStaticProjection(hidden_dim)
82     self.filter = sentiment_filter
83     self.stage_detector = StageDetector() if use_stage_sep else None
84     self.lambda_val = lambda_val
85     self.use_stage_sep = use_stage_sep
86
87     def get_effective_lambda(self, step: int, local_entropy: float) -> float:
88         """根据生成阶段计算有效回响强度。"""
89         if not self.use_stage_sep:
90             return self.lambda_val
91         stage = self.stage_detector.detect(step, local_entropy)
92         return self.lambda_val if stage == 'thinking' else 0.5 * self.lambda_val
93
94     def process_step(
95         self,
96         hidden_states: torch.Tensor,
97         chosen_idx: int,
98         step: int,
99         local_entropy: float = 0.0
100     ) -> torch.Tensor:
101         """处理单步解码：回收丢弃Token 筛选 投影 注入。"""
102         # 构建丢弃集
103         mask = torch.ones(hidden_states.size(0), dtype=torch.bool)
104         mask[chosen_idx] = False
105         discarded = hidden_states[mask]
106
107         # 情感筛选（可选）
108         if self.filter is not None:
109             batch = [
110                 (str(i), discarded[i])
111                 for i in range(discarded.size(0))
112             ]
113             filtered = self.filter.filter(batch)
114             if filtered:
115                 vecs = torch.stack([f[1] for f in filtered])
116                 wts = torch.tensor([f[2] for f in filtered])
117                 centroid = (vecs * wts.view(-1, 1)).sum(0) / wts.sum()
118             else:
119                 centroid = discarded.mean(0)
120         else:
121             centroid = discarded.mean(0)
122
123         # 随机静态投影
124         echo = self.proj.project(centroid)
125
126         # 计算有效强度（阶段分离）
127         effective_lambda = self.get_effective_lambda(step, local_entropy)

```

```

128
129     # 注入
130     chosen_h = hidden_states[chosen_idx]
131     return chosen_h + effective_lambda * echo.to(chosen_h.device)

```

A.4 实验配置和超参数完整列表

表 11 列出了所有实验中使用的超参数及其取值。

表 11: 超参数完整列表

类别	参数	取值
模型	名称	Qwen2.5-0.5B-Instruct
	隐藏维度 d_{model}	896
	词汇表大小 $ \mathcal{V} $	151,936
	注意力头数	16
	Transformer 层数	24
解码	$\text{top-}p$	0.9
	温度 T	0.7
	最大生成长度	200 Token
	重复惩罚	1.0 (关闭)
回响	回响强度 λ	{0, 0.5, 1.0, 2.0}
	衰减率 γ	0.9
	情感阈值 τ	0.1
	思考比例 T_{think}	0.15
硬件	GPU	NVIDIA A100 80G
	CUDA 版本	12.1
	PyTorch 版本	2.1.0
评估	提示词数量	15 (5 维度 $\times$ 3 条)
	重复次数	3
	随机种子	42

实验环境说明：所有实验在单张 NVIDIA A100 80G GPU 上执行。模型使用 bfloat16 精度加载以节省显存。由于语义回响需要在每个解码步访问所有候选 Token 的隐藏状态（约 544MB/步），A100 的 80G 显存足以容纳模型权重（约 1GB）和回响池的临时计算开销。在基线（ $\lambda = 0$ ）配置下，每个提示词的平均生成时间约为 2.3 秒；在 $\lambda = 0.5$ 配置下，平均生成时间约为 21.8 秒（约 9.5 倍基线），主要开销来自质心计算和情感筛选。